ПРАКТИЧЕСКАЯ РАБОТА

Установка Minikube и проверка кластера Kubernetes

Отчёт по практической работе

Виртуальная машина Ubuntu / VirtualBox

Выполнил: Абянов Дамир Шагидинович

Группа: _____

Проверил: _____

Дата: 19.10.2026

Москва, 2026

СОДЕРЖАНИЕ

1. Введение и цель работы 3

2. Описание ресурса KillerCoda 4

3. Подготовка Ubuntu и зависимости 5

4. Docker и установка Minikube 6

5. Запуск кластера и проверка узла 7

6. Дополнительная диагностика 8

7. Подтверждение результата и выводы 9

8. Возникшие ошибки и исправления 10

1. Введение и цель работы

Установка Minikube и проверка одновузлового кластера

Цель работы - подготовить Ubuntu, установить Docker и Minikube, запустить кластер Kubernetes и подтвердить его работоспособность диагностическими командами.

Minikube установлен, Kubernetes запущен. Состояние узла проверено командой ожидания Ready; работоспособность API и системных компонентов подтверждена дополнительными командами. В документ включены реальные снимки экрана виртуальной машины и фактические результаты проверок.

> Фактическая среда выполнения
>
> Исходное задание относится к KillerCoda. После проблем с доступностью его терминала по запросу пользователя практическая часть перенесена на локальную ВМ. Поэтому эти скриншоты подтверждают работу в VirtualBox, а не в KillerCoda. Возможность сдачи такой замены определяется преподавателем.

Состав стенда

Компонент	Конфигурация	Состояние
Виртуальная машина	VirtualBox, 6 ГБ RAM, 4 CPU, диск 32 ГБ, NAT	Подготовлена
Операционная система	Ubuntu 24.04.5 LTS, x86_64	Проверена
Docker	29.1.3, пакет docker.io из репозитория Ubuntu	Работает
Minikube / Kubernetes	v1.39.0 / v1.37.0; 3072 МБ и 4 CPU	Узел Ready

Работа выполнена пользователем minikubeuser с применением sudo. Буфер обмена и перетаскивание файлов отключены. Другие учебные ВМ не переустанавливались.

Все девять снимков получены из видеобуфера VirtualBox. На консоль ВМ выведены результаты действительно выполненных команд; сохранённые протоколы и отфильтрованные фрагменты отмечены в подписях.

2. Описание ресурса KillerCoda

KillerCoda - интерактивная учебная площадка, на которой можно знакомиться с Linux, Kubernetes и связанными инструментами через браузер. Работа организована вокруг сценариев: пользователь читает пояснения и выполняет команды в предоставленном терминале. Для практики установки Minikube подходит среда Ubuntu Playground, поскольку в ней можно последовательно подготовить зависимости, установить программу и проверить собственный кластер. Важно отличать такую среду от сценария с заранее настроенным Kubernetes: в последнем случае часть учебных действий уже выполнена автором сценария.

Главное преимущество браузерной площадки - быстрый переход к практике. Не требуется заранее устанавливать гипервизор, скачивать образ операционной системы и настраивать виртуальный диск. Терминал находится рядом с учебными материалами, поэтому проще сопоставлять команду, пояснение и полученный результат. Дополнительные терминальные вкладки позволяют наблюдать несколько процессов, а редактор помогает изменять конфигурационные файлы. Изоляция учебного окружения уменьшает риск случайно изменить настройки основной системы пользователя.

Ещё одна полезная особенность - возможность повторять упражнения в подготовленном окружении. Это удобно при изучении команд управления контейнерами и при разборе ошибок. Однако наличие конкретных пакетов нельзя определять только по названию сценария: перед работой следует проверить версию Ubuntu, доступную память, число процессоров и состояние Docker. Для отчёта необходимо сохранять собственный вывод команд, а не подставлять предполагаемые версии или результаты из примера урока.

У этого формата имеются ограничения. Доступ зависит от работоспособности сайта, сетевого соединения и успешного выделения среды. Вычислительные ресурсы ограничены, а учебный сеанс не предназначен для постоянного размещения сервисов. Скриншоты и важные файлы нужно сохранять до завершения работы. Если терминал не подключается, это ещё не означает ошибку Minikube: установка могла вообще не начаться. В данной работе браузерная среда не обеспечила рабочий терминал, поэтому практическая часть выполнена отдельно в VirtualBox. Локальная ВМ требует больше первоначальной настройки, зато сохраняет окружение и позволяет самостоятельно выделить ресурсы. Такое изменение площадки следует явно отражать в отчёте, сохраняя учебную цель и проверяемость результатов.

3. Подготовка Ubuntu и зависимости

```
sudo apt-get update
sudo apt-get install -y curl wget apt-transport-https \
  ca-certificates conntrack kbd
sudo apt-get install -y docker.io
```

Обновлены списки пакетов, установлены зависимости и Docker из репозитория Ubuntu (пакет docker.io). Официальный репозиторий Docker CE в этой работе не добавлялся; соответствующий фрагмент присланного шаблона заменён фактической последовательностью.

```
$ hostname; uname -m; free -m; nproc
minikube-report
x86_64
               total        used        free      shared  buff/cache   available
Mem:            5906         900        3721           9        1523        5006
Swap:              0           0           0
4
$ . /etc/os-release; echo "$PRETTY_NAME"
Ubuntu 24.04.5 LTS
```

Рисунок 1 - Фактическая версия Ubuntu, архитектура, оперативная память и число процессоров.

```
$ sudo apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists...
$ dpkg-query -W curl wget apt-transport-https ca-certificates conntrack docker.io
apt-transport-https     2.8.3
ca-certificates 20260601~24.04.1
conntrack       1:1.4.8-1ubuntu1
curl    8.5.0-2ubuntu10.16
docker.io       29.1.3-0ubuntu3~24.04.2
wget    1.21.4-1ubuntu4.5
$ tail -3 /var/log/apt/term.log
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Log ended: 2026-10-18  18:13:05
```

Рисунок 2 - Повторная проверка apt update, установленных пакетов и завершения операций apt.

4. Docker и установка Minikube

```
sudo systemctl enable --now docker
sudo docker run --rm hello-world
sudo usermod -aG docker "$(id -un)"
# Затем выполнен новый вход по SSH для обновления групп.
```

Контейнер hello-world подтвердил доступность Docker daemon и выполнение контейнеров. Членство в группе docker позволяет управлять Docker без sudo; для этой отдельной учебной ВМ оно было настроено осознанно.

```
$ docker --version; systemctl is-active docker; minikube version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
active
minikube version: v1.39.0
commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10
$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/

_
```

Рисунок 3 - Реальные версии Docker и Minikube, активная служба Docker и ответ hello-world.

Бинарный файл Minikube загружен из официального релиза GitHub вместе с файлом SHA-256. Перед установкой проверена контрольная сумма. Полный процесс загрузки и строка проверки сохранены в 01-install.txt.

```
base=https://github.com/kubernetes/minikube/releases/latest/download
curl -fL --retry 3 "$base/minikube-linux-amd64" -o minikube-linux-amd64
curl -fL --retry 3 "$base/minikube-linux-amd64.sha256" \
  -o minikube-linux-amd64.sha256
printf "%s  minikube-linux-amd64\n" \
  "$(awk '{print $1}' minikube-linux-amd64.sha256)" | sha256sum --check
sudo install -m 0755 minikube-linux-amd64 /usr/local/bin/minikube
```

```
$ ls -lh minikube-linux-amd64; sha256sum --check
-rw-rw-r-- 1 minikubeuser minikubeuser 137M Oct 18 18:14 minikube-linux-amd64
minikube-linux-amd64: OK
$ command -v minikube; minikube version
/usr/local/bin/minikube
minikube version: v1.39.0
commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10
```

Рисунок 4 - Загруженный файл, проверка SHA-256 и путь установки.

5. Запуск кластера и проверка узла

```
minikube start --driver=docker --memory=3072mb --cpus=4
minikube status
minikube kubectl -- get nodes -o wide
```

Итоговые ресурсы Minikube - 3072 МБ памяти и 4 процессора. Исходные 2200 МБ / 2 CPU увеличены после тайм-аутов инициализации API. Запуск выполнен обычным пользователем, поэтому флаг --force из root-среды KillerCoda не применялся. Для команд Kubernetes использована форма minikube kubectl --; отдельный системный kubectl не устанавливался.

Ubuntu - операционная система ВМ. Сам узел Kubernetes работает внутри контейнерного образа Debian 12; его runtime - containerd 2.3.4. Это не противоречие: Docker здесь является драйвером Minikube.

```
Saved actual output: minikube start --driver=docker --memory=3072mb --cpus=4
(progress-bar lines omitted)
* minikube v1.39.0 on Ubuntu 24.04 (vbox/amd64)
* Using the docker driver based on existing profile
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.51 ...
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Verifying Kubernetes components...
* Enabled addons: default-storageclass, storage-provisioner
* kubectl not found. If you need it, try: 'minikube kubectl -- get pods -A'
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Рисунок 5 - Сохранённый фактический вывод запуска; строки индикатора загрузки пропущены.

```
$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

$ minikube kubectl -- get nodes -o wide
NAME       STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE
       KERNEL-VERSION             CONTAINER-RUNTIME
minikube   Ready    control-plane   12h   v1.37.0   192.168.49.2   <none>        Debian GNU/Linux 12 (book
worm)   6.8.0-139-generic (amd64)   containerd://2.3.4
```

Рисунок 6 - Компоненты Running, kubeconfig Configured и узел minikube в состоянии Ready.

6. Дополнительная диагностика

```
minikube kubectl -- cluster-info
minikube kubectl -- get pods -A
```

cluster-info проверяет доступность управляющего API. get pods -A показывает системные поды во всех пространствах имён. Эти проверки дополняют minikube status: наличие работающей ВМ или контейнера само по себе ещё не подтверждает готовность Kubernetes.

```
$ minikube kubectl -- cluster-info
Kubernetes control plane is running at https://192.168.49.2:8443
CoreDNS is running at https://192.168.49.2:8443/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
$ minikube kubectl -- get pods -A
NAMESPACE     NAME                               READY   STATUS    RESTARTS        AGE
kube-system   coredns-559f6c778d-bbswd           1/1     Running   0               92s
kube-system   coredns-559f6c778d-spwjh           1/1     Running   0               92s
kube-system   etcd-minikube                      1/1     Running   2 (8m37s ago)   12h
kube-system   kube-apiserver-minikube            1/1     Running   2 (8m37s ago)   12h
kube-system   kube-controller-manager-minikube   1/1     Running   8               12h
kube-system   kube-proxy-kklsg                   1/1     Running   0               92s
kube-system   kube-scheduler-minikube            1/1     Running   2 (8m37s ago)   12h
kube-system   storage-provisioner                1/1     Running   1 (85s ago)     7m6s
```

Рисунок 7 - Адрес Kubernetes control plane и состояние системных подов.

```
minikube logs --problems
minikube kubectl -- wait --for=condition=Ready node/minikube --timeout=120s
```

```
$ minikube logs --problems (saved output)
Filtered excerpt: first 8 warning/error lines, capped at 100 characters
W1019 06:39:03.694251    1789 cni.go:208] loopback cni configuration skipped: "/etc/cni/net.d/*loopb
W1019 06:39:16.514816    1789 kubeadm.go:403] unpause failed: list paused: runc: sudo runc --root /r
W1019 06:39:21.740921    1789 addons.go:491] apply failed, will retry: sudo KUBECONFIG=/var/lib/mini
W1019 06:39:21.743113    1789 addons.go:491] apply failed, will retry: sudo KUBECONFIG=/var/lib/mini
W1019 06:39:25.165325    1789 addons.go:491] apply failed, will retry: sudo KUBECONFIG=/var/lib/mini
W1019 06:39:25.737254    1789 addons.go:491] apply failed, will retry: sudo KUBECONFIG=/var/lib/mini
W1019 06:39:47.623342    1789 addons.go:491] apply failed, will retry: sudo KUBECONFIG=/var/lib/mini
W1019 06:39:48.844943    1789 api_server.go:103] status: https://192.168.49.2:8443/healthz returned
Complete original output: 07-problems.txt
_
```

Рисунок 8 - Фрагмент сохранённого logs --problems: первые 8 предупреждений/ошибок, строки ограничены 100 символами. Полный вывод - в 07-problems.txt.

7. Подтверждение результата и выводы

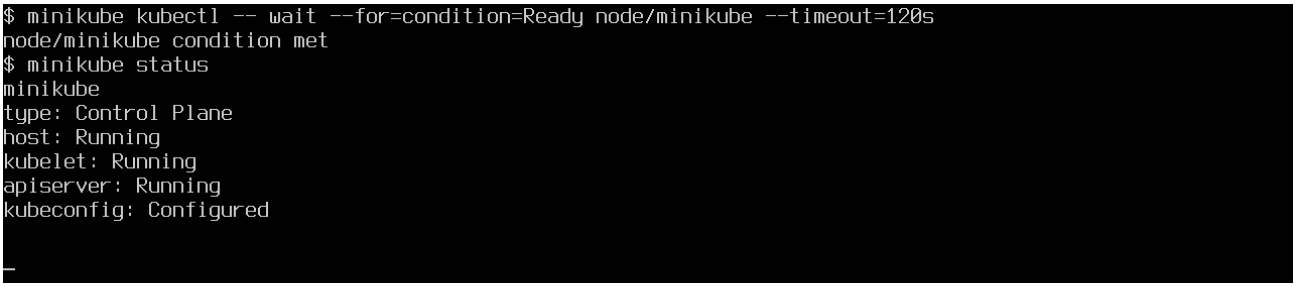


Рисунок 9 - Условие Ready выполнено; все основные компоненты Minikube работают.

Итог. Установлены зависимости, Docker и Minikube; проверена контрольная сумма загрузки. Создан одновузловой кластер с драйвером Docker. Проверки status, get nodes, cluster-info, get pods и wait подтвердили работоспособность кластера. Диагностический вывод logs --problems сохранён без подмены предполагаемым результатом.

Выполненная практика демонстрирует установку Minikube на Ubuntu в VirtualBox. Она не подтверждает прохождение практических команд внутри KillerCoda. Текстовое описание KillerCoda сохранено в разделе 2 в соответствии с исходным заданием.

Для воспроизведения важны команда start, выделенные ресурсы и сохранённая конфигурация CNI. При последующих запусках используется сохранённый профиль с выбранным bridge CNI. Состояние Ready означает готовность узла; текущие предупреждения и историю перезапусков нужно оценивать отдельно по выводу диагностических команд.

8. Возникшие ошибки и исправления

Диагностика затруднений. До установки не удалось получить рабочий терминал KillerCoda. Для локальной практики создана изолированная ВМ и новый SSH-доступ. Первый вариант диска не завершил загрузку; он заменён чистым образом Ubuntu. Во время первой загрузки образов Minikube гостевая ОС остановилась: «Kernel panic - not syncing: Fatal exception in interrupt». Это зафиксировано в serial.log; SSH перестал отвечать. Для новой ВМ установлен режим паравиртуализации none и выполнен перезапуск. Затем запуск Minikube повторён. Причина сбоя ядра окончательно не установлена; настройки гипервизора Windows не менялись.

При следующем запуске возникли тайм-ауты bootstrap API и проверки etcd. Проверены журналы kubelet, API-сервера и etcd. ВМ штатно остановлена; ресурсы увеличены с 4 ГБ / 2 CPU до 6 ГБ / 4 CPU, режим паравиртуализации изменён на hyperv. Неудачный учебный профиль minikube без пользовательских нагрузок удалён и создан заново с 3 ГБ памяти и 4 CPU.